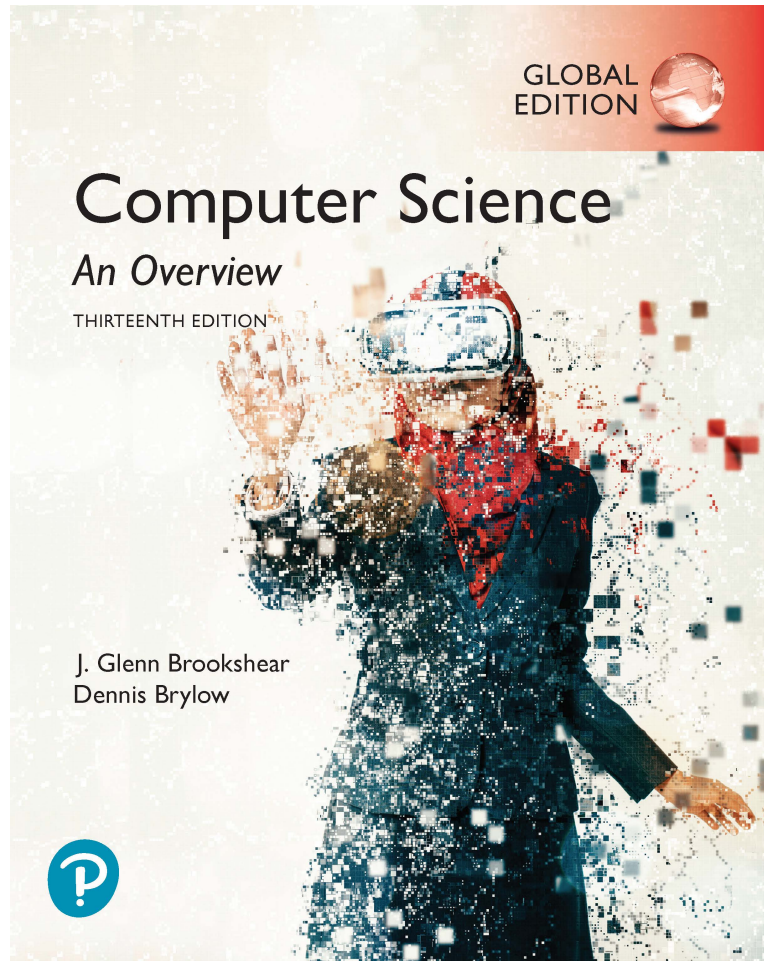


Computer Science An Overview

13th Edition, Global Edition



Chapter 5 Algorithms

Chapter 5: Algorithms

- 5.1 The Concept of an Algorithm 概念
- 5.2 Algorithm Representation 呈現
- 5.3 Algorithm Discovery 设计
- 5.4 Iterative Structures 迴圈結構
- 5.5 Recursive Structures 遞迴結構
- 5.6 Efficiency and Correctness 效率、正確性

5.1 The Concept of an Algorithm

- Algorithms from previous chapters
 - Converting from one base to another
 - Correcting errors in data
 - Compression
- Many researchers believe that every activity of the human mind is the result of an algorithm



Formal Definition of Algorithm

有序的集合

不模糊的

- An algorithm is an ordered set of unambiguous, executable steps that defines a terminating process
- The steps of an algorithm can be sequenced in different ways
 - Linear (1, 2, 3, ...) 線性
 - Parallel (multiple processors) 平行
 - Cause and Effect (circuits)

Formal Definition of Algorithm

- A Terminating Process 最终导向一个终止条件
 - Culminates with a result
 - Can include systems that run continuously
 - Hospital systems
 - Long Division Algorithm
- A Non-terminating Process NP Hard Problem
 - Does not produce an answer
 - Chapter 12 : “Non-deterministic Algorithms”



The Abstract Nature of Algorithms

- There is a difference between an algorithm and its representation.
 - Analogy: difference between a story and a book
- A Program is a representation of an algorithm.
- A Process is the activity of executing an algorithm.

Program: 演算法的一種表示法

Process: 一種執行演算法的活動

5.2 Algorithm Representation

正式

定義明確的

基礎

• Is done formally with well-defined Primitives ⇒ 一個定義良好 (定義充足) 的基礎要件

— A collection of primitives constitutes a programming language.

非正式

• Is done informally with Pseudocode

— Pseudocode is between natural language and a programming language.

Figure 5.2 Folding a bird from a square piece of paper

P. 264

好強 2^{2^2}

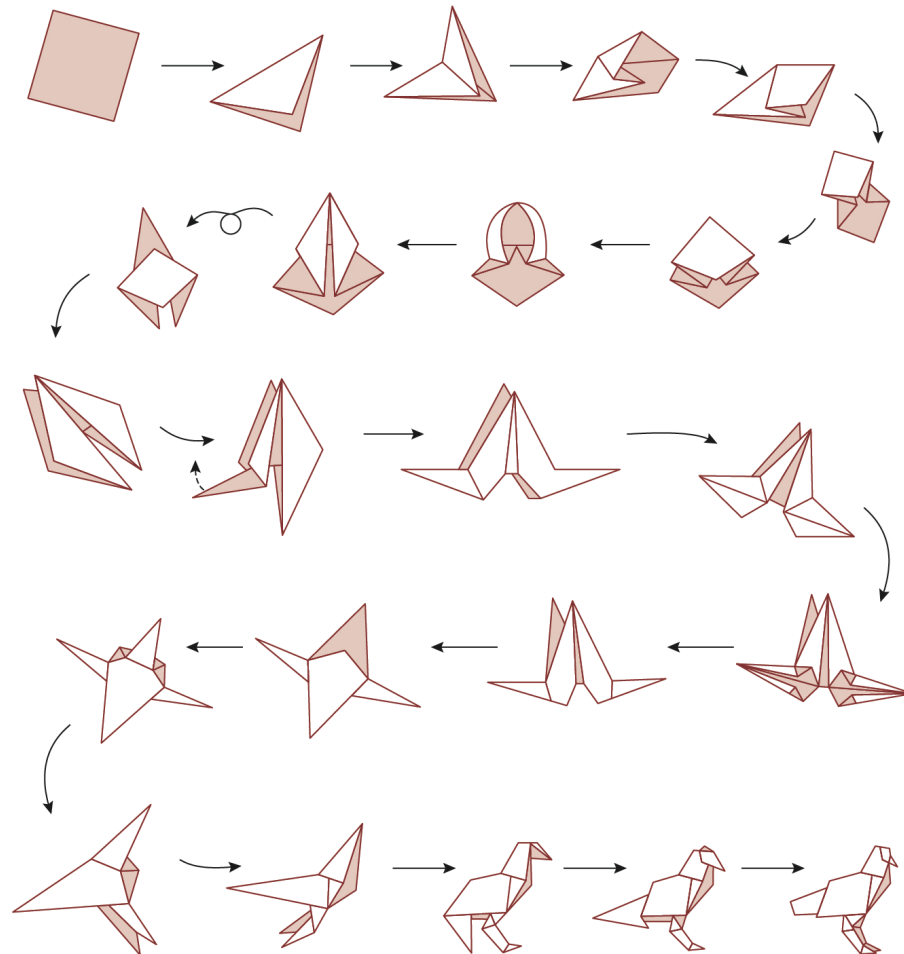
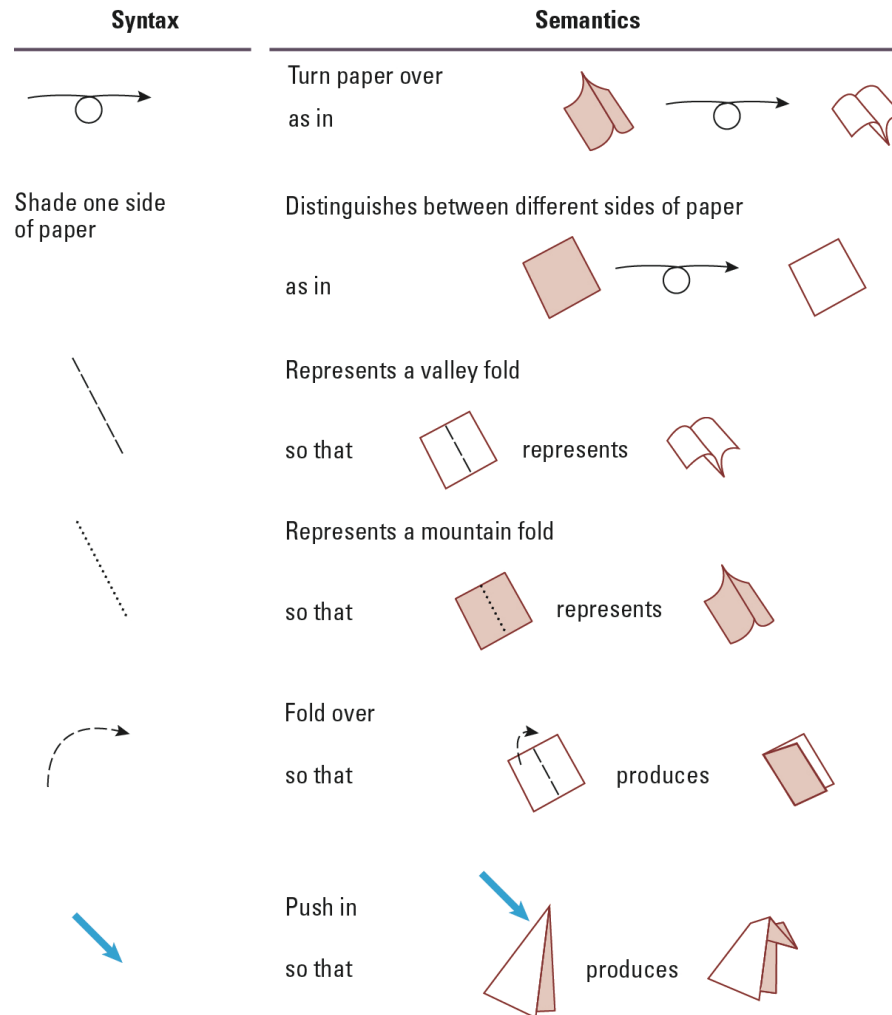


Figure 5.3 Origami primitives



Designing a Pseudocode Language

- Choose a common programming language 選定一個程式語言
- Loosen some of the syntax rules 放鬆語法的限制
- Allow for some natural language 加上自然語言註解
- Use consistent, concise notation 從頭到尾使用一致、簡潔的規則
- We will use a Python-like Pseudocode

Pseudocode Primitives

- Assignment

name = *expression*

- example

RemainingFunds = CheckingBalance +
SavingsBalance

Pseudocode Primitives (continued)

- Conditional selection

if (*condition*):
 activity

- example

← 加入自然语言

if (sales have decreased):
 lower the price by 5%

Pseudocode Primitives (continued)

- Conditional selection

if (*condition*):

activity

else:

activity

- example

if (year is leap year):

daily total = total / 366

else:

daily total = total / 365

Pseudocode Primitives (continued)

- Repeated execution

while (*condition*):
 body

- example

while (tickets remain to be sold):
 sell a ticket

Pseudocode Primitives (continued)

- Indentation shows **nested** conditions

if (not raining):

 if (temperature == hot):

 go swimming

 else:

 play golf

else:

 watch television

Pseudocode Primitives (continued)

- Define a function

```
def name():
```

- example

```
def ProcessLoan():
```

- Executing a function

```
if (...):
```

```
    ProcessLoan()
```

```
else:
```

```
    RejectApplication()
```

Figure 5.4 The procedure Greetings in pseudocode

```
def Greetings():  
    Count = 3  
    while (Count > 0):  
        print('Hello')  
        Count = Count - 1
```

Pseudocode Primitives (continued)

- Using parameters

```
def Sort(List):
```

```
    .
```

```
    .
```

- Executing Sort on different lists

```
Sort(the membership list)
```

```
Sort(the wedding guest list)
```

5.3 Algorithm Discovery 演算法的發掘

- The first step in developing a program
- More of an art than a skill
- A challenging task

Polya's 解題策略

Polya's Problem Solving Steps

口訣:

→ 理 設 執 驗
解 計 行 證

- 1. Understand the problem. 理解問題
- 2. Devise a plan for solving the problem. 設計解決問題的方法
- 3. Carry out the plan. 嘗試、執行想法、計劃
- 4. Evaluate the solution for accuracy and its potential as a tool for solving other problems. 驗證正確性, 並挖掘可能可以被此演算法解決的潛在問題。

Polya's Problem Solving:

1. Understand the Problem
2. Devise a Plan
3. Carry out Plan
4. look back

Polya's Steps in the Context of Program Development

- 1. Understand the problem.
- 2. Get an idea of how an algorithmic function might solve the problem.
- 3. 正規化、公式化 Formulate the algorithm and represent it as a program.
- 4. Evaluate the solution for accuracy and its potential as a tool for solving other problems. 驗證正確性,並挖掘可能可以被此演算法解決的潛在問題。

Getting a Foot in the Door

- Try working the problem backwards 嘗試把問題「反著做」
- Solve an easier related problem
 - Relax some of the problem constraints (先把原先問題的條件放寬)
 - Solve pieces of the problem first (bottom up methodology) (Top down)
(把問題分解成更簡單的問題)
- Stepwise refinement: Divide the problem into smaller problems (top-down methodology)

Ages of Children Problem

- Person A is charged with the task of determining the ages of B's three children.
 - B tells A that the product of the children's ages is 36.
 - A replies that another clue is required.
 - B tells A the sum of the children's ages.
 - A replies that another clue is needed.
 - B tells A that the oldest child plays the piano.
 - A tells B the ages of the three children.
- How old are the three children?

Figure 5.5 p. 274

a. Triples whose product is 36

$(1,1,36)$	$(1,6,6)$
$(1,2,18)$	$(2,2,9)$
$(1,3,12)$	$(2,3,6)$
$(1,4,9)$	$(3,3,4)$

b. Sums of triples from part (a)

$1 + 1 + 36 = 38$
$1 + 2 + 18 = 21$
$1 + 3 + 12 = 16$
$1 + 4 + 9 = 14$

$1 + 6 + 6 = 13$
$2 + 2 + 9 = 13$
$2 + 3 + 6 = 11$
$3 + 3 + 4 = 10$

5.4 Iterative Structures 迴圈結構

- A collection of instructions repeated in a looping manner
- Examples include:
 - Sequential Search Algorithm 循序搜尋演算法
(線性搜尋法)
 - Insertion Sort Algorithm (插入排序法)

Figure 5.6 The sequential search algorithm in pseudocode

```
def Search (List, TargetValue):  
    if (List is empty):  
        Declare search a failure  
    else:  
        Select the first entry in List to be TestEntry  
        while (TargetValue > TestEntry and entries remain):  
            Select the next entry in List as TestEntry  
        if (TargetValue == TestEntry):  
            Declare search a success  
        else:  
            Declare search a failure
```

Figure 5.7 Components of repetitive control

Initialize: Establish an initial state that will be modified toward the termination condition
初始化
(初始步驟、條件)

Test: Compare the current state to the termination condition and terminate the repetition if equal
測試、比較
與終止條件比較

Modify: Change the state in such a way that it moves toward the termination condition
修改
改變狀態，
使其朝向終止條件 (terminate condition) 前進

Iterative Structures

- Pretest loop: 預測試迴圈

`while (condition):` ← 優先檢查迴圈條件
`body` cond 為 True 時 成立, 才執行 `body` 的內容
 為進入條件

- Posttest loop: 後測試迴圈

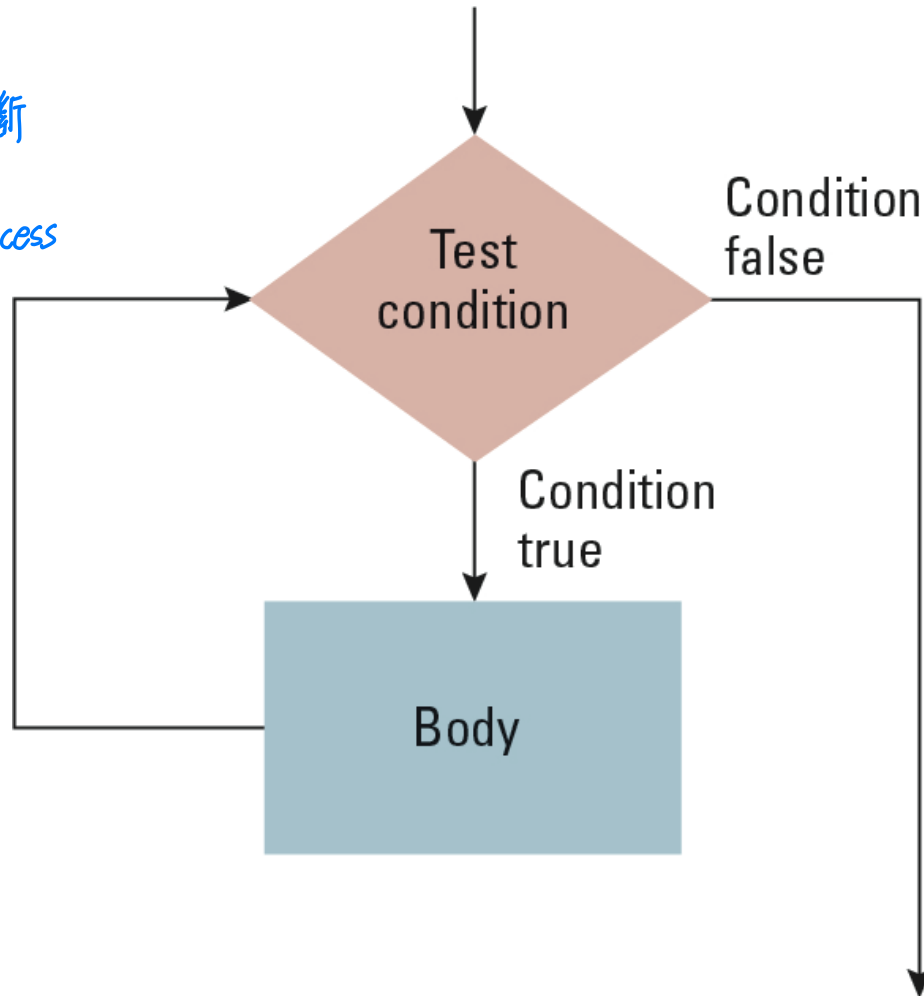
`repeat:`
`body` ← 一定至少先做一次 `body` 的內容
`until (condition)` 再來檢查迴圈條件
 ↓
 cond 為 True 時
 是離開條件, 不是進入條件

Pretest-loop 前測回圈

Figure 5.8 The while loop structure

◇ 菱形：一个判断

□ 矩形：一个 Process



Posttest-loop 後測迴圈

Figure 5.9 The repeat loop structure

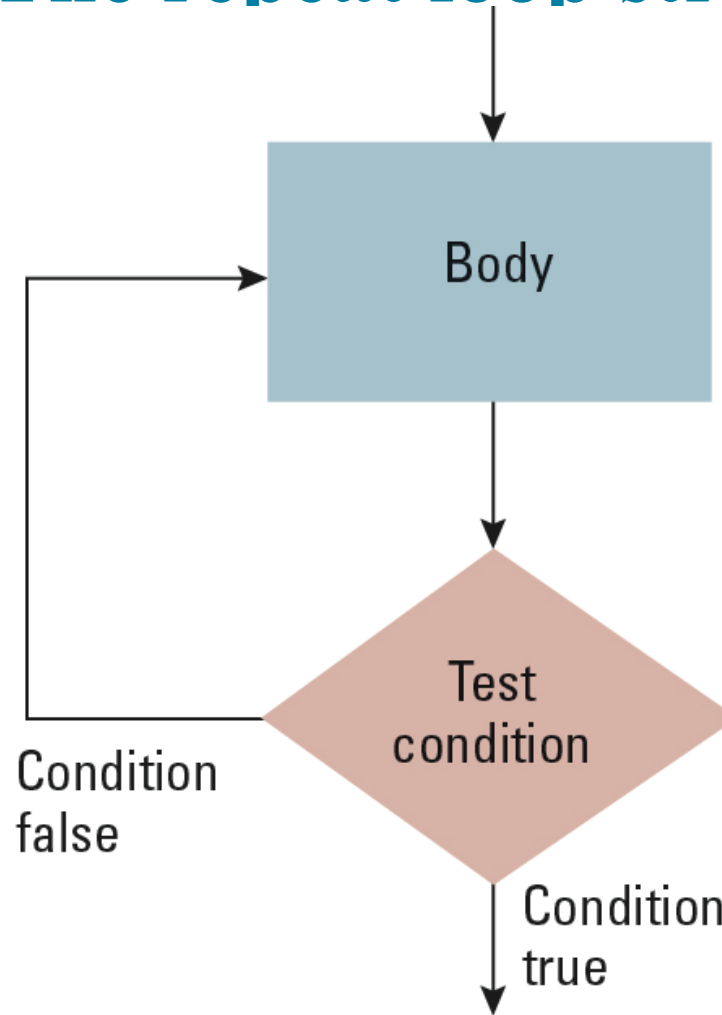


Figure 5.10 Sorting the list Fred, Alex, Diana, Byron, and Carol alphabetically

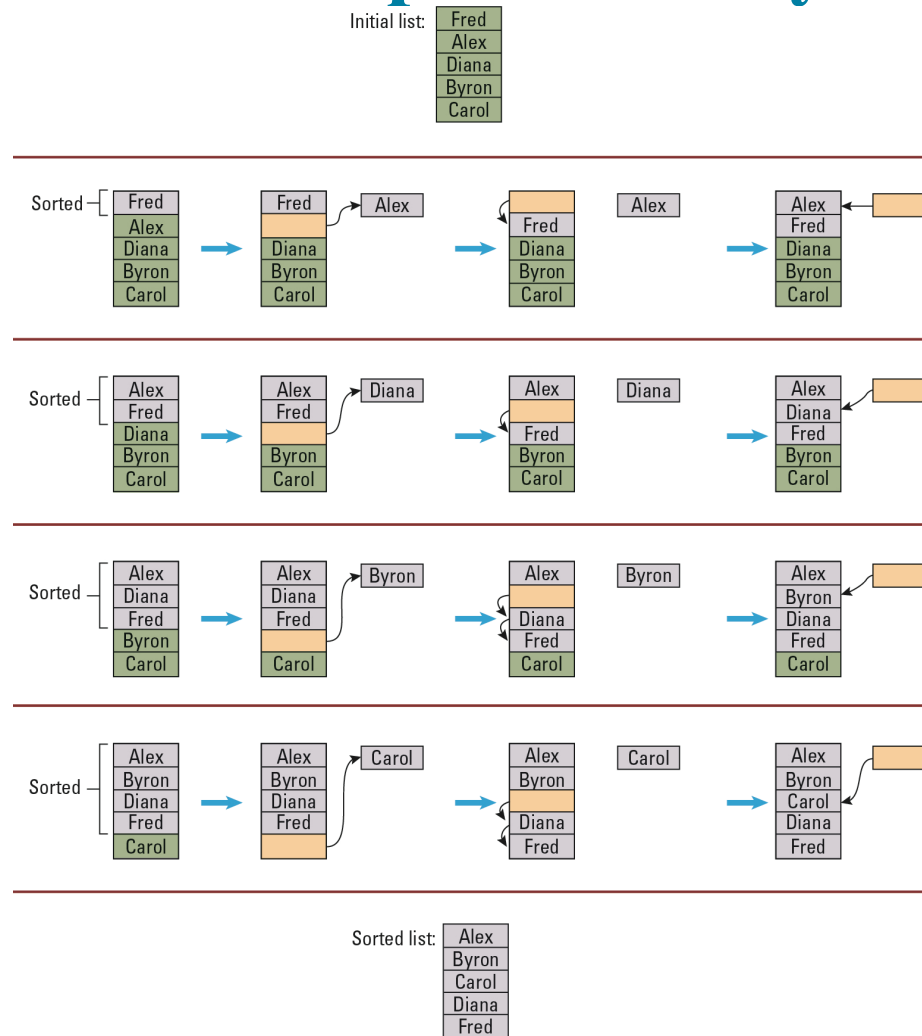


Figure 5.11 The insertion sort algorithm expressed in pseudocode

```
def Sort(List):
```

```
    N = 2
```

```
    while (N <= length of List):
```

```
        Pivot = Nth entry in List
```

```
        Remove Nth entry leaving a hole in List
```

```
        while (there is an Entry above the  
                hole and Entry > Pivot):
```

```
            Move Entry down into the hole leaving  
            a hole in the list above the Entry
```

```
        Move Pivot into the hole
```

```
    N = N + 1
```

5.5 Recursive Structures

遞迴結構

- Repeating the set of instructions as a subtask of itself.
- Multiple activations of the procedure are formed, all but one of which are waiting for other activations to complete. → (呼叫堆疊)
- Example: The Binary Search Algorithm

$$\log_2 n$$

Figure 5.12 Applying our strategy to search a list for the entry John

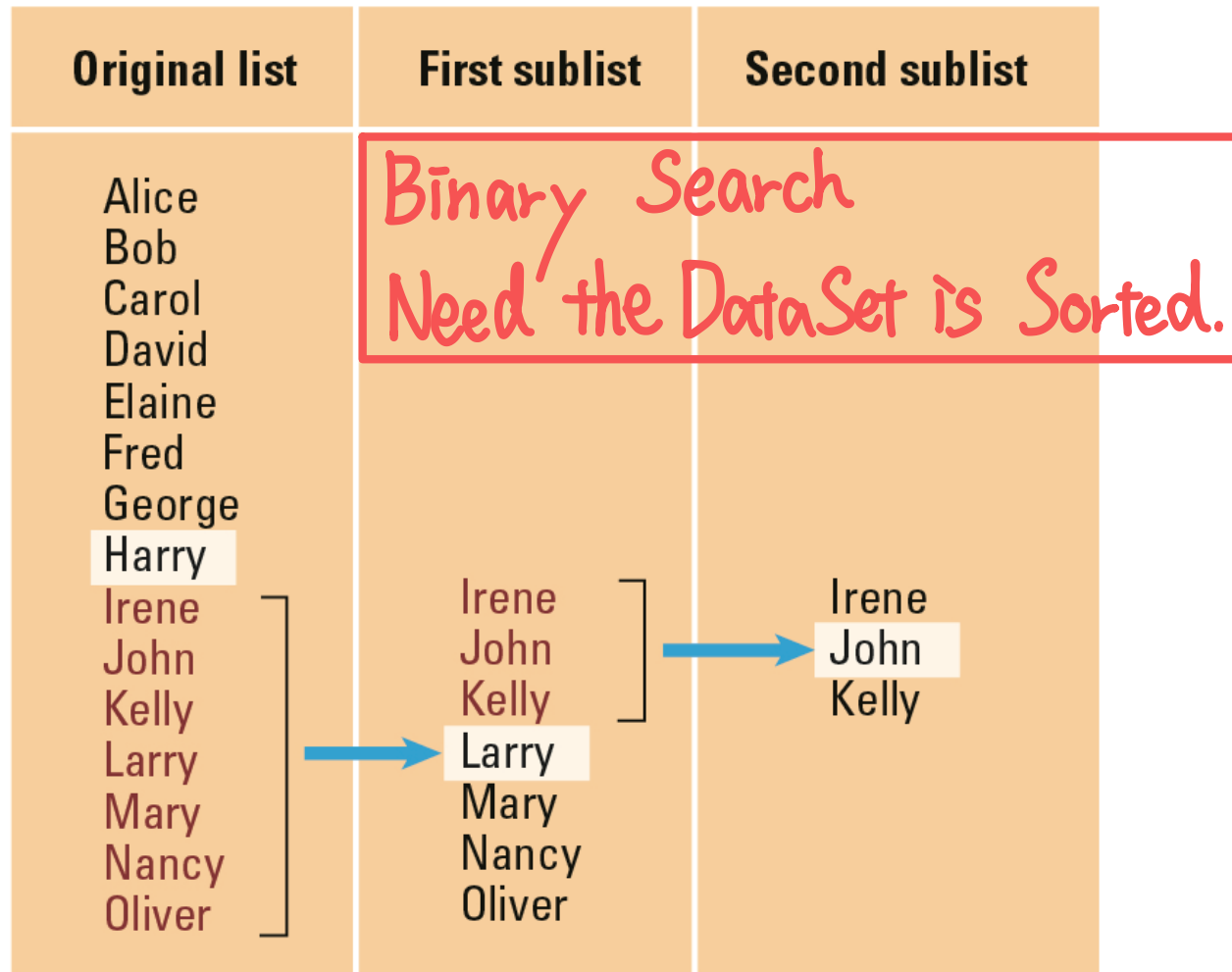


Figure 5.13 A first draft of the binary search technique

if (List is empty):

 Report that the search failed

else: *課P.291 Paragraph 2 last line:*

 TestEntry = middle entry in the List

 if (TargetValue == TestEntry):

 Report that the search succeeded

 if (TargetValue < TestEntry):

 Search the portion of List preceding TestEntry for
 TargetValue, and report the result of that search

 if (TargetValue > TestEntry):

 Search the portion of List following TestEntry for
 TargetValue, and report the result of that search

*"In this case, let us agree that the
 'middle' entry refers to the first entry in the
 second half of the list."*

Figure 5.14 The binary search algorithm in pseudocode

```
def Search(List, TargetValue):  
    if (List is empty):  
        Report that the search failed  
    else:  
        TestEntry = middle entry in the List  
        if (TargetValue == TestEntry):  
            Report that the search succeeded  
        if (TargetValue < TestEntry):  
            Sublist = portion of List preceding TestEntry  
            Search(Sublist, TargetValue)  
        if (TargetValue > TestEntry):  
            Sublist = portion of List following TestEntry  
            Search(Sublist, TargetValue)
```

Figure 5.15 Recursively Searching

We are here.

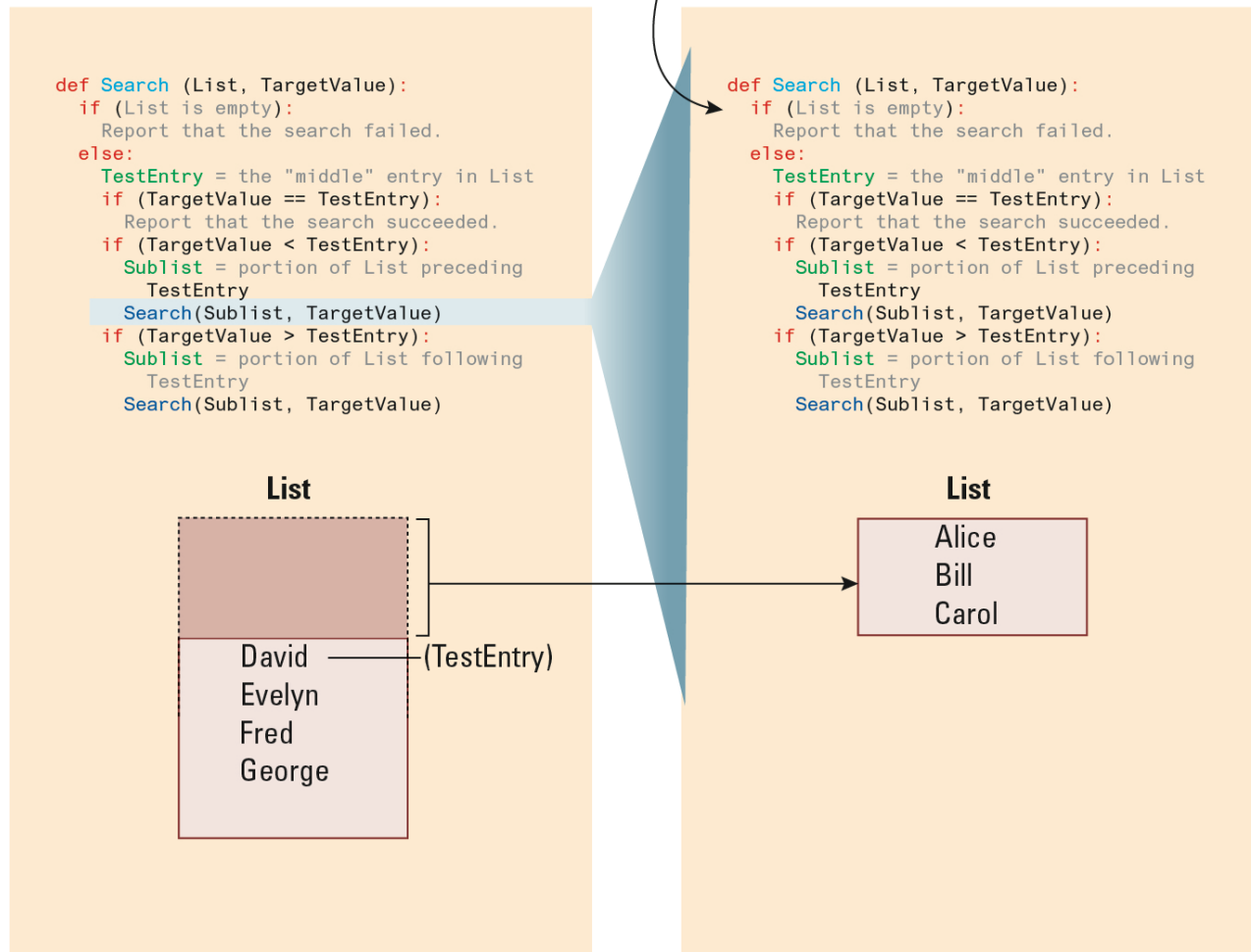


Figure 5.16 Second Recursive Search

We are here.

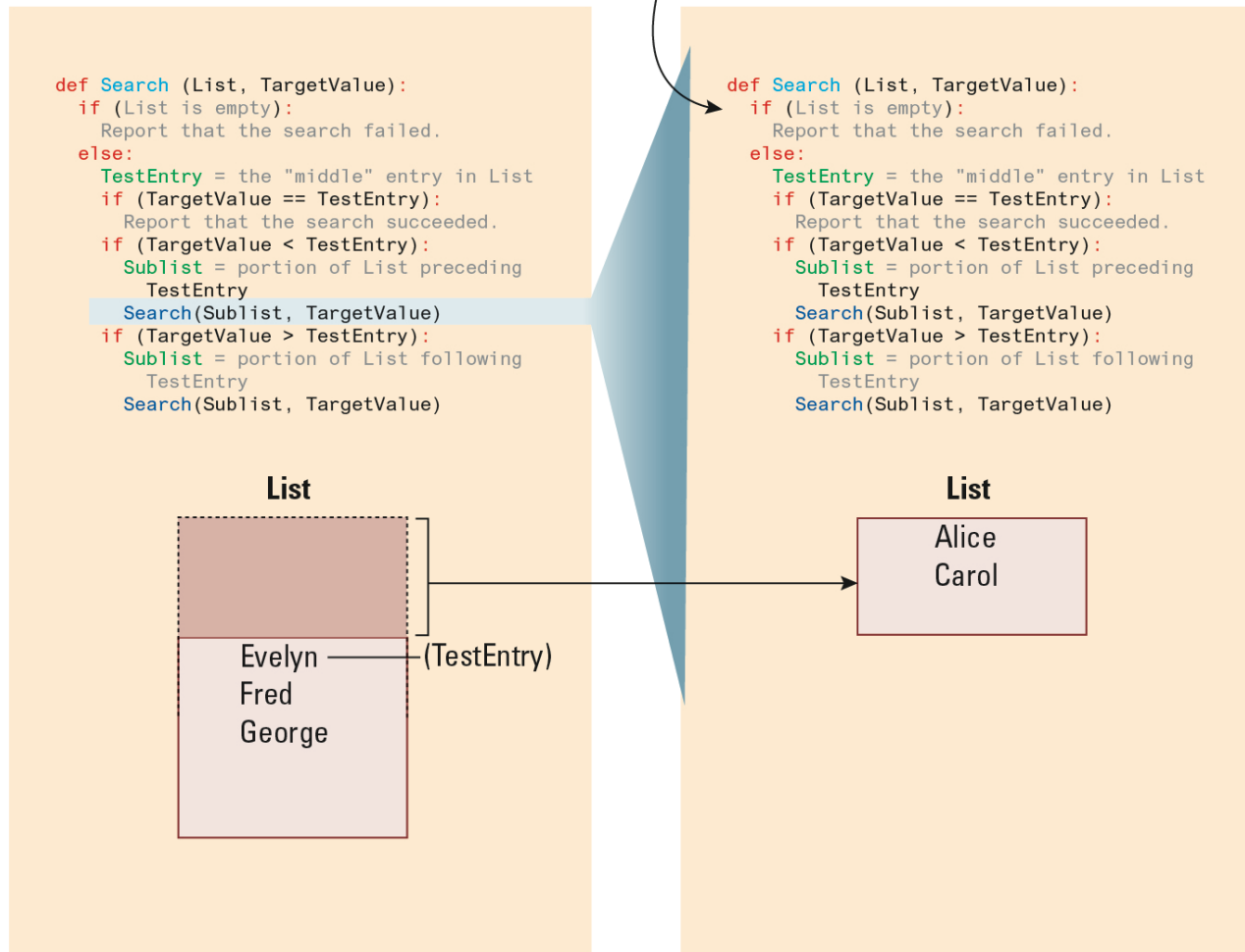
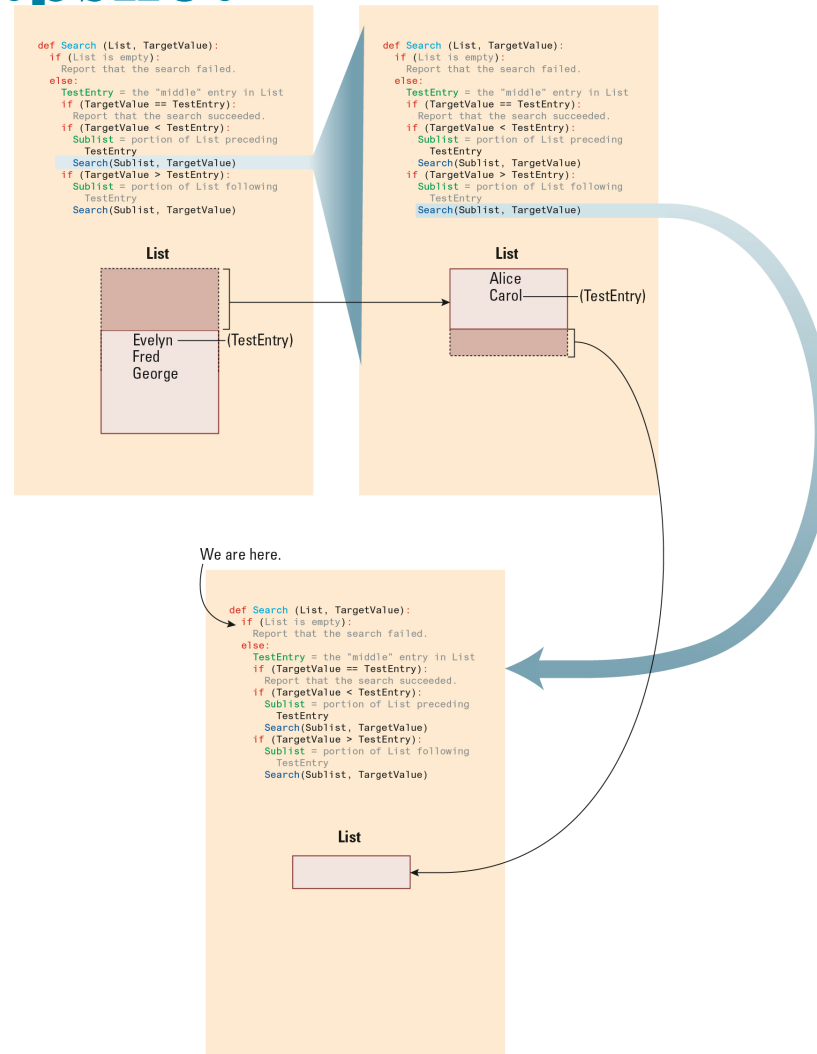


Figure 5.17 Second Recursive Search, Second Snapshot



Recursive Control

要求 初始化 修改

- Requires initialization, modification, and a test for termination (base case)
終止條件 基 值
- Provides the illusion of multiple copies of the function, created dynamically in a telescoping manner
- Only one copy is actually running at a given time, the others are waiting

其他Process 都在呼叫堆疊內

5.6 Efficiency and Correctness

- The choice between efficient and inefficient algorithms can make the difference between a practical solution and an impractical one
- The correctness of an algorithm is determined by reasoning formally about the algorithm, not by testing its implementation

Efficiency

／ 去衡量其要執行指令的數量

- Measured as number of instructions executed
- Uses ^{or big O} **big theta** notation:
 - Example: Insertion sort is in $\Theta(n^2)$ Binary Search: $\log_2 n$
- Incorporates best, worst, and average case analysis
包含最佳、最差 & 平均 case

*P300 ~ P.302

Figure 5.18 Applying the insertion sort in a worst-case situation

Comparisons made for each pivot					
Initial list	1st pivot	2nd pivot	3rd pivot	4th pivot	Sorted list
Elaine David Carol Barbara Alfred	1 → Elaine David Carol Barbara Alfred	3 → David 2 → Elaine Carol Barbara Alfred	6 → Carol 5 → David 4 → Elaine Barbara Alfred	10 → Barbara 9 → Carol 8 → David 7 → Elaine Alfred	Alfred Barbara Carol David Elaine
	第一次：比一次	比二次	比三次	比四次……	

最差情況下

N 个 element, 會比較 $\frac{1+2+\dots+(n-1)}{2} = \frac{n^2-n}{2} \Rightarrow O(n^2)$ Worst Case

Average: $\frac{n^2-n}{4} \Rightarrow O(n^2)$

Figure 5.19 Graph of the worst-case analysis of the insertion sort algorithm

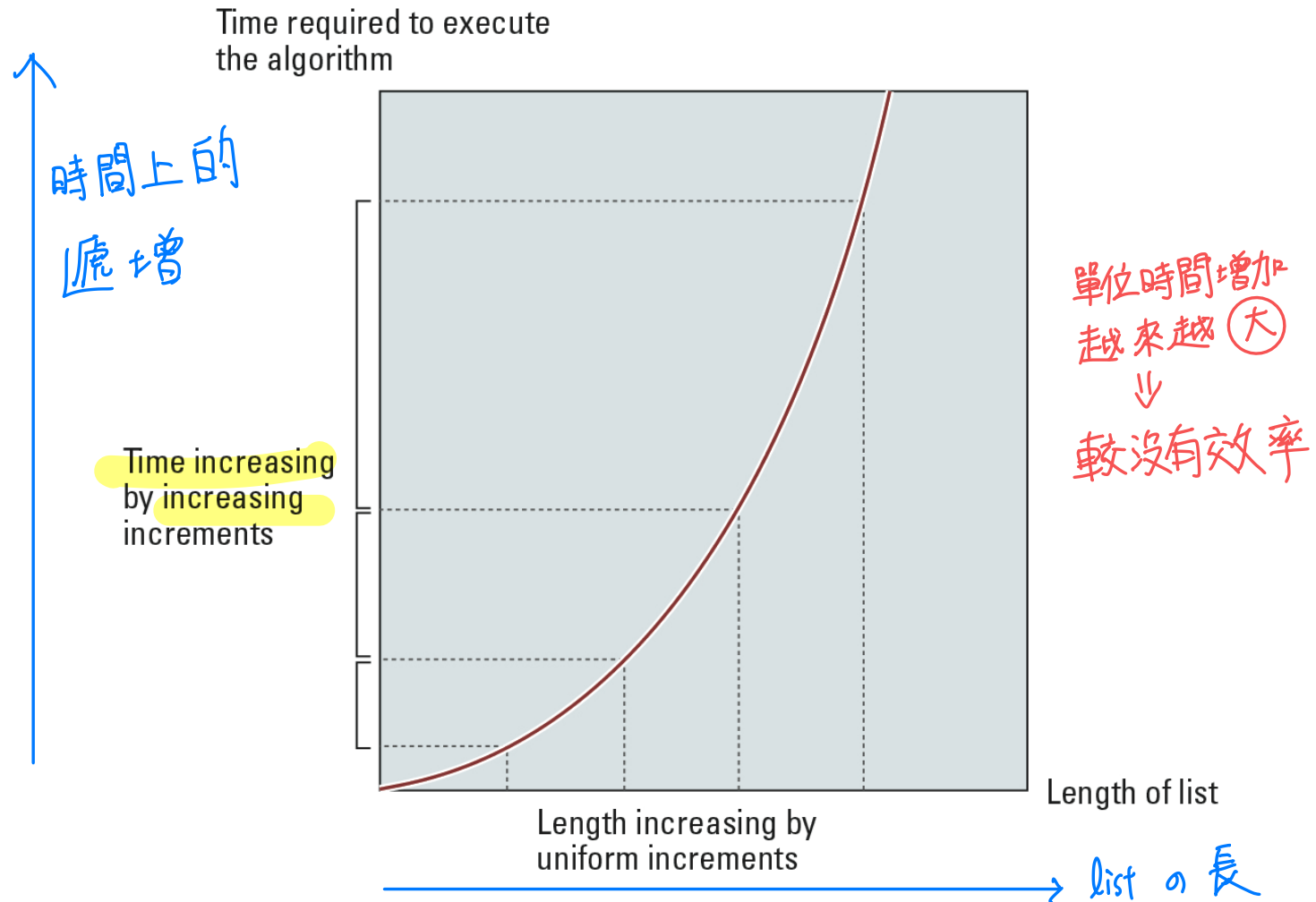
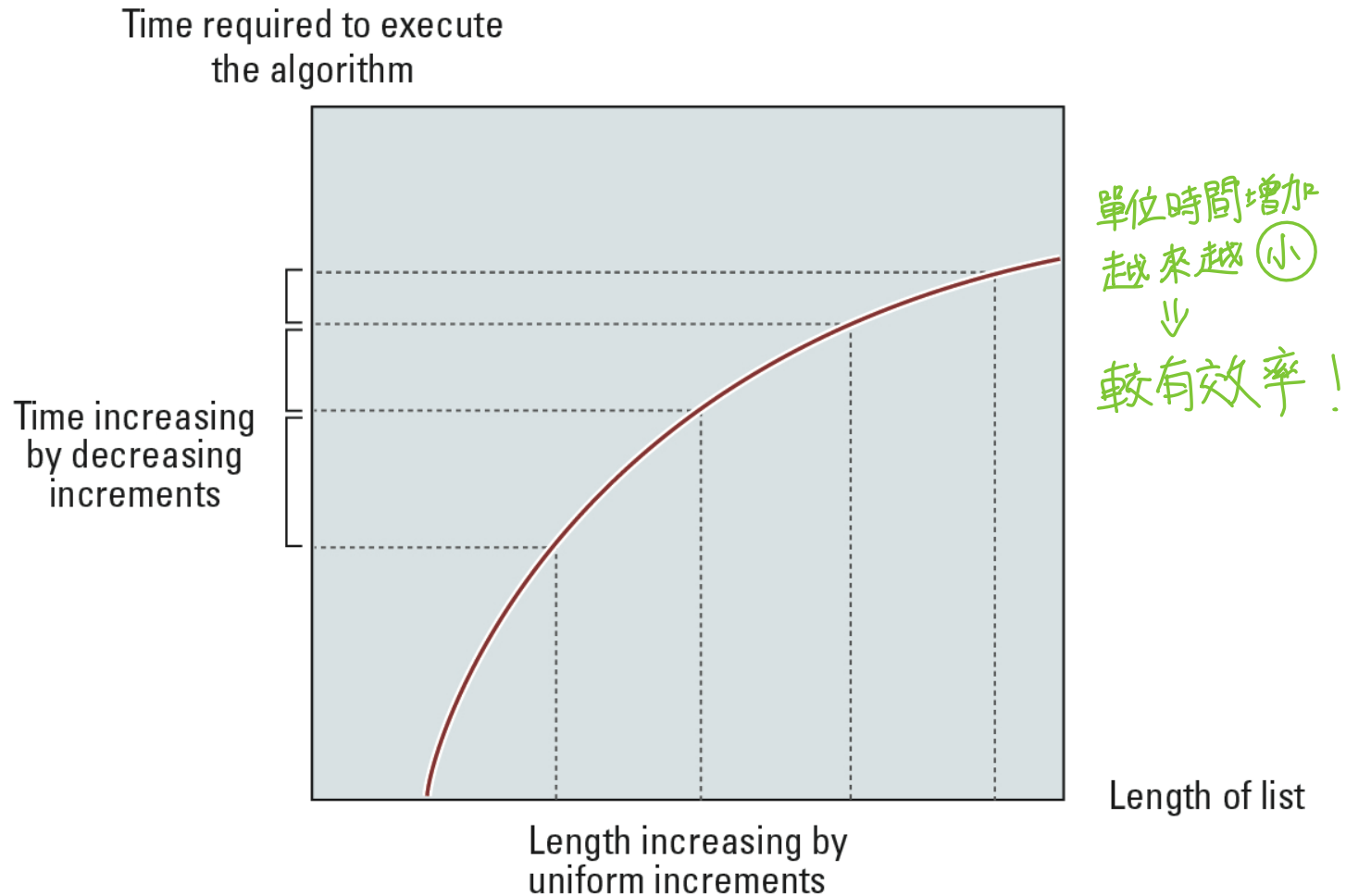


Figure 5.20 Graph of the worst-case analysis of the binary search algorithm



Software Verification

P.306

軟體的驗證
(驗證演算法的正確性)

- Proof of correctness (with formal logic) 從邏輯上驗證
- Assertions 斷言、主張
 - 圈外 → Preconditions 每次進回圈之前必定成立的事實 / Post Cond. 每次出回圈之後必成立的事實
 - 圈內 → Loop invariants 每次回圈開始之前一定會成立的事實
- Testing is more commonly used to verify software
- Testing only proves that the program is correct for the test cases used

Chain Separating Problem

- A traveler has a gold chain of seven links.
- He must stay at an isolated hotel for seven nights.
- The rent each night consists of one link from the chain.
- What is the fewest number of links that must be cut so that the traveler can pay the hotel one link of the chain each morning without paying for lodging in advance?

Figure 5.21 Separating the chain using only three cuts

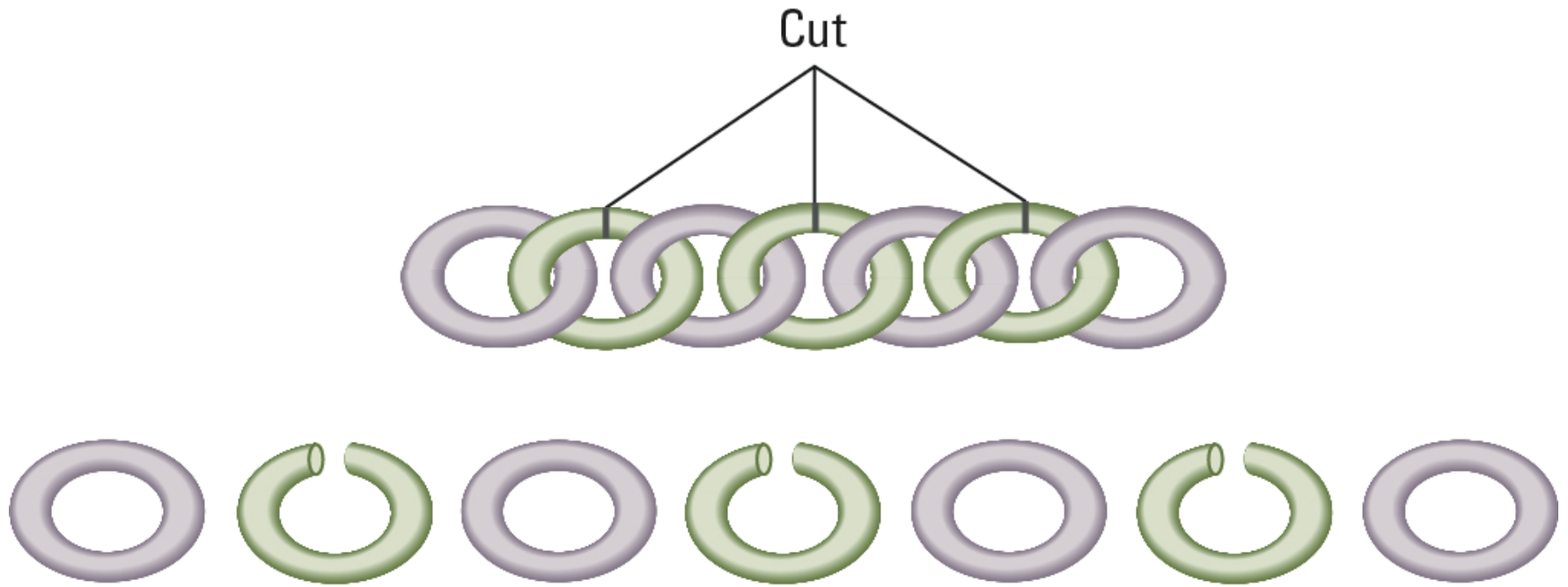


Figure 5.22 Solving the problem with only one cut

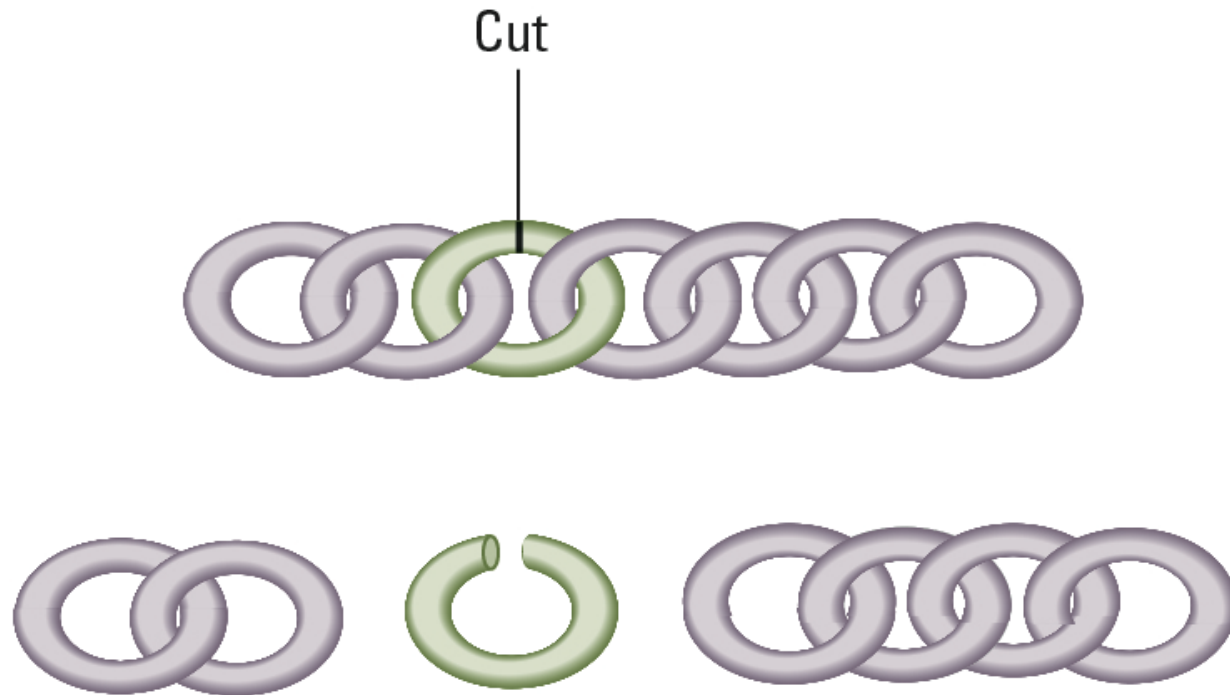


Figure 5.23 The assertions associated with a typical while structure

